qual_by_position.py 사용법과 설명

lab11/src/qual_by_position.py는 FASTQ 파일을 읽어 read 위치(1번째 염기, 2번째 염기, …)별 평균 품질 점수(Phred)를 출력하는 스크립트이다. 표준 라이브러리(sys)만 사용한다. FASTQ를 읽는 read_fastq 함수는 parse_fastq.py, qual_summary.py의 것과 같다.

qual_summary.py가 파일 전체의 평균 품질을 숫자 하나로 보여준다면, 이 스크립트는 read의 어느 위치에서 품질이 떨어지는지 보여준다.

FASTQ 파일의 모든 read에서 같은 위치(1번째 염기, 2번째 염기, …)의 품질 점수를 모아 위치별 평균을 계산하고, 위치와 평균 품질을 탭으로 구분하여 출력한다(그 위치의 염기 개수 count도 함께 출력한다). 이것이 FastQC의 Per base sequence quality 그래프가 계산하는 것과 같은 값이다.

1. 사용법

lab 폴더(lab11/)에서 FASTQ 파일 경로를 인자로 주고 실행한다. 결과는 화면(표준 출력)에 탭으로 구분된 표(TSV)로 출력된다.

```
cd lab11
python3 src/qual_by_position.py data/SRR1553607_1.fastq
```

인자	설명
sys.argv[1]	읽을 FASTQ 파일 경로 (필수)

결과를 파일로 저장하려면 리다이렉션(>)을 사용한다.

```
python3 src/qual_by_position.py data/SRR1553607_1.fastq > doc/qual_by_position_1.tsv
```

출력 형식

열	설명
position	read 안의 위치 (1부터 시작)
count	그 위치까지 길이가 닿는 read의 수 (그 위치의 염기 개수)
mean_quality	그 위치 염기들의 평균 Phred 점수 (소수점 둘째 자리 반올림)

실행 결과 예시

```
$ python3 src/qual_by_position.py data/SRR1553607_1.fastq
position count mean_quality
1 203445 32.04
2 203445 32.17
3 203445 32.28
4 203445 35.85
5 203445 35.65
...
99 203445 29.03
100 203445 28.93
101 203445 28.25
```

출력 중 일부 위치만 골라 보면 다음과 같다.

위치	SRR1553607_1	SRR1553607_2	trimmed
1	32.04	30.03	32.28
4	35.85	33.72	36.07
14~15 (최고)	38.47 (14)	36.12 (15)	38.93 (14)
50	33.44	30.57	34.92
101 (최저)	28.25	25.78	30.17
Q30 미만 위치 수	15개 (87~101)	48개 (54~101)	0개

실습 데이터(약 20만 read, 101bp)에서 실행 시간은 약 7초이다.

2. 동작 원리

1. read_fastq로 모든 read를 (이름, 서열, 품질) 튜플의 리스트로 읽는다.
2. 위치별 품질 점수 합계 totals와 염기 개수 counts를 리스트로 둔다. 인덱스 i가 위치 i + 1에 해당한다.
3. 각 read의 품질 문자열을 한 글자씩 보면서 ord(문자) - 33(Phred+33)으로 점수를 구해 totals[i]에 더하고 counts[i]를 1 늘린다.
4. 지금까지 본 read보다 긴 read가 나오면 두 리스트 끝에 0을 추가해 늘린다. 그래서 read 길이를 미리 알 필요가 없다.
5. 마지막에 위치마다 totals[i] / counts[i]를 계산해 출력한다.

```
for name, seq, qual in reads:
    for i in range(len(qual)):
        if i >= len(totals):
            totals.append(0)
            counts.append(0)
        totals[i] = totals[i] + ord(qual[i]) - 33
        counts[i] = counts[i] + 1
```

길이가 다른 read 처리

위치마다 따로 counts를 세기 때문에 trimming으로 read 길이가 제각각이어도 올바른 평균이 나온다. 예를 들어 길이 3과 5인 read가 있으면 위치 1~3은 두 read의 평균, 위치 4~5는 긴 read 하나의 값이다. 모든 read를 같은 수(전체 read 수)로 나누면 뒤쪽 위치의 평균이 실제보다 낮게 계산되므로 주의한다.

trimmed.fastq에서는 위치 1~50의 count가 188,357로 모든 read와 같고, 그 뒤로 점점 줄어 위치 101에서는 171,032이다. fastp가 50bp보다 짧아진 read를 버렸기 때문에 모든 read가 최소 50bp이다.

3. 결과 해석

- 앞쪽 1~3번 위치: 4번 위치보다 약 3~4 낮다. Illumina 시퀀싱의 처음 몇 cycle에서 흔히 보이는 현상이다.
- 뒤쪽으로 갈수록 감소: 14~15번 위치에서 최고점을 찍은 뒤 cycle이 진행될수록 품질이 꾸준히 떨어진다. 시퀀싱 화학 반응이 반복되면서 오류가 쌓이기 때문이다.
- read 1과 read 2: _2 파일은 모든 위치에서 _1보다 낮고, 54번 위치부터 Q30 아래로 떨어진다.

qual_summary.py에서 본 전체 평균 차이(약 2.6)가 주로 뒤쪽 위치에서 생긴다는 것을 알 수 있다.

- trimming 효과: fastp로 품질이 낮은 3' 끝을 잘라낸 trimmed.fastq는 모든 위치가 Q30 이상이다. 위치 101의 평균이 28.25에서 30.17로 올랐다.

4. 주의사항과 한계

- 메모리 사용: readlines()는 파일 전체를 메모리에 올린다. 실습 데이터에는 문제가 없지만 수 GB 크기의 FASTQ에는 한 줄씩 읽는 방식이 적합하다.
- 형식 검사 없음: 모든 read가 정확히 4줄이라고 가정한다. 줄 수가 4의 배수가 아니면 IndexError가 난다.
- 인자 누락: 파일 경로 없이 실행하면 sys.argv[1]에서 IndexError가 난다.
- 빈 파일: read가 하나도 없으면 헤더 줄만 출력하고 끝난다.
- 위치별 평균의 한계: 뒤쪽 위치는 긴 read만으로 계산되므로 count가 작은 위치의 평균은 소수 read에 크게 좌우될 수 있다. count 열을 함께 확인한다.
- 인코딩 가정: Phred+33만 처리한다. Phred+64 데이터에서는 점수가 31만큼 높게 계산된다.
- 압축 파일 미지원: .fastq.gz는 직접 읽을 수 없으므로 gunzip으로 먼저 압축을 풀어야 한다.

5. 활용 예

출력이 TSV이므로 다른 명령어와 연결하기 쉽다.

```
# 평균 품질이 Q30 미만인 위치만 보기
python3 src/qual_by_position.py data/SRR1553607_2.fastq ¦ awk -F'\t' 'NR > 1 && $3 < 30'

# 평균 품질이 가장 높은 위치 찾기
python3 src/qual_by_position.py data/SRR1553607_1.fastq ¦ tail -n +2 ¦ sort -t$'\t' -k3,3gr ¦ head -1

# trimming 전후 비교: 두 결과를 나란히 붙이기
python3 src/qual_by_position.py data/SRR1553607_1.fastq > before.tsv
python3 src/qual_by_position.py data/trimmed.fastq > after.tsv
paste before.tsv after.tsv ¦ cut -f1,3,6
```

같은 정보는 FastQC 보고서(doc/SRR1553607_1_fastqc.html, doc/trimmed_fastqc.html)의 "Per base sequence quality" 그래프에서도 확인할 수 있다. FastQC는 뒤쪽 위치를 몇 개씩 묶어(예: 35-39) 보여주고 평균 외에 사분위수도 그린다는 점이 다르다.